Research Project — AIDEA

Generalist Structured Extraction of Italian Transport Documents and Invoices: A Controlled Benchmark of OCR, Layout-Parsing, and Vision–Language Pipelines

Jan Wangrat[1]*, Vadim Caracuianu[1], Shilpa Rajeev[1] and Federico Romeo[2]

[1]*Politecnico di Milano* [2]*AIDEA*

This manuscript was compiled on May 28, 2026

Abstract

Converting business documents into structured records is a recurring industrial need, yet production systems still rely on per-vendor templates that break on unseen layouts. We study the template-free alternative for the ChatDDT project: extracting validated JSON from Italian transport documents (*documenti di trasporto*, DDT) and invoices. We build a modular benchmark of *fifteen* pipelines over fifty DDTs and twenty invoices, in which twelve share one language model, prompt, and target schema so that only the document front-end varies, and three hosted schema-guided pipelines are run end to end as monolithic baselines (LlamaExtract in cost-effective and agentic tiers, plus Google's Document AI Custom Extractor). The front-ends span native vision–language extraction, local and cloud OCR, layout-to-Markdown converters, table detection, and hosted extraction. The leading pipelines—a LlamaParse-to-Markdown or Surya-OCR front-end feeding the model—reach roughly 93% field accuracy at about a cent per document; native vision–language extraction on the raw PDF is statistically level on field accuracy at under half the cost. Structure-heavy front-ends (table recognition, layout converters, agentic tiers) consistently underperform clean reading-order text, and the two commercial OCR services we add (Mistral OCR and Google's document OCR) match native extraction at best. The residual error concentrates in per-line order-reference fields and is dominated by omission rather than hallucination. Bootstrap confidence intervals show the leading pipelines are tied on accuracy, so under the project's stated priority order—generalization, accuracy, robustness, cost, latency—the choice falls to the lower-ranked axes. We recommend the cost-effective LlamaParse-to-Markdown front-end feeding the model because, among the tied leaders, its Markdown doubles as auditable source text for review and downstream document linking; local Surya OCR is the equally accurate, lower-cost in-house alternative, and native extraction remains a defensible baseline.

Keywords: *document AI, structured extraction, OCR, layout parsing, vision–language models, benchmark, invoices*

1. INTRODUCTION

Businesses exchange a steady stream of semi-structured paper: orders, transport documents, and invoices that encode the same commercial facts in as many layouts as there are suppliers. Turning that paper into queryable records is the entry point for reconciliation, auditing, and analytics. The dominant production approach—per-vendor templates that read fixed coordinates—is brittle: a new supplier, a reflowed table, or a scanned page silently breaks it. The ChatDDT project, commissioned by AIDEA, asks the opposite question: how far can a single *template-free* pipeline go on heterogeneous Italian business documents, and which building blocks are worth their cost?

1.1. Motivation and context

The Italian procurement cycle links three document types. An *order* states intent; one or more *documenti di trasporto* (DDT, transport/delivery notes) accompany the physical shipment and typically carry no pricing; and one or more *invoices* (*fatture*) request payment with full pricing and terms. A single order may fan out into several DDTs, and several DDTs are often consolidated into one monthly invoice. The present work targets the first and load-bearing capability: reading individual DDTs and invoices into validated JSON. Reconstructing the cross-document links is a deliberate downstream goal and is out of scope here.

1.2. Problem statement

The task is to map a PDF—assumed worst case to be a scanned image—to a JSON object conforming to a fixed DDT or invoice schema (§3), with no supplier-specific rules and no "field is always at coordinate (x, y)" assumptions. The client fixes an explicit priority order that we adopt as our evaluation lens throughout: *generalization* first, then *accuracy*, *robustness*, *cost*, and *latency*. The methodology is left open—OCR, multimodal models, vision–language models, hybrids—under two conditions: the chosen strategy must be explainable step by step, and it must come with a cost estimate.

1.3. Contributions

The space of candidate building blocks is large and the literature offers little guidance on which combination suits sparse commercial tables. We therefore treat the design question empirically and contribute:

- a **modular benchmark harness** that separates an expensive, cached *prediction* stage from a cheap *scoring* stage, so a new pipeline or a new metric can be added without recomputing the rest (§6);
- a **controlled comparison** in which twelve pipelines share one language model, prompt, and target schema so that only the document front-end varies—making the parser the single independent variable—alongside two hosted schema-guided services (LlamaExtract in two tiers; Google's Document AI Custom Extractor) evaluated end to end as external baselines (§5);
- **cost and latency accounting** at real, published rates, computed per document from logged token and page counts

*Corresponding author.

We benchmark fifteen document-extraction pipelines on Italian transport documents (DDT) and invoices for the ChatDDT project. Code, the evaluation harness, and per-document results are available in the project repository.

(§4);

- an **analysis** that localizes the residual error to a small set of per-line reference fields, shows it to be omission rather than hallucination, and yields a concrete deployment recommendation under the client's priorities (§8–§10).

2. BACKGROUND AND RELATED WORK

Document extraction has converged on three broad families, and our pipelines are drawn from each so that the comparison spans the realistic design space rather than a single school.

OCR and layout parsing. Classical systems first recover text and geometry, then impose structure. Surya [14] is a modern open OCR stack (text detection, recognition, reading order, layout, and table recognition) competitive with cloud OCR while running locally. Layout-to-Markdown converters go further and emit a structured document: Docling [8] and Marker [10] assemble headings, paragraphs, and tables into Markdown, the latter optionally invoking a language model to repair difficult regions. Table structure can also be recovered directly by detection models; the Table Transformer [5], a DETR detector [1] trained on PubTables-1M, localizes tables and their cell grids.

Schema-guided extraction services. A second family hides parsing behind a hosted API that returns structured data against a user schema. LlamaParse converts documents to Markdown at several quality tiers, and LlamaExtract performs schema-constrained extraction on top of it [21]; Google Document AI offers pre-trained processors (a generic Form Parser and a specialized Invoice parser) that emit typed entities [19]. These trade transparency and unit cost for turnkey convenience.

Vision–language models. The third family skips explicit parsing: a multimodal model reads the page image and emits structure in one step. Schema-constrained decoding lets such models return JSON conforming to a target schema directly. We use Gemini [6] in this mode. The approach descends from OCR-free document understanding—Donut [4] and Nougat [7]—and from layout-aware encoders such as LayoutLMv3 [3]; recent open OCR-centric models such as DeepSeek-OCR [18] and dots.ocr [20] sit between the OCR and vision–language families. Beyond Gemini the current frontier spans both proprietary multimodal models and strong open-weight ones—Qwen2.5-VL [24] and InternVL [9] report document-understanding parity with the leading closed models; we deliberately hold the extraction model fixed (§5) to isolate the front-end and revisit this choice as a limitation (§10).

Positioning. Our aim is not a new model but a controlled measurement: given one capable extraction model, which front-end—if any—earns its cost on sparse Italian commercial documents? We therefore reuse the components above as interchangeable front-ends behind a fixed extraction back-end, and report accuracy together with the operational quantities the client cares about.

3. DATA AND TASK

3.1. Corpus

The benchmark uses a confidential corpus of **50 DDTs** and **20 invoices**, each PDF paired with a hand-authored ground-truth JSON. The DDTs span more than twenty issuing companies and a wide range of layouts—bordered and borderless item tables, single- and multi-page, digital and scanned—while the invoices come from two vendors with markedly different templates. The set is small by machine-learning standards but representative of the heterogeneity the pipeline must absorb, and large enough to separate the methods reliably at the field level (thousands of scored line-item cells).

3.2. Target schemas

The extraction target is a fixed JSON schema per document type, validated with Pydantic [13]. A DDT carries document metadata, the sending and receiving parties, and a list of line items; pricing fields are optional because delivery notes frequently omit them:

```
{ "document": {"type":"DDT","number":...,"date":...},
  "sender":   {"name": ...},
  "receiver": {"name": ...},
  "items": [ {"code":...,"description":...,
              "quantity":...,"unit_of_measure":...,
              "unit_price":null,"total_amount":null} ] }
```

The invoice schema adds a document total, currency, and a `payment` block; its line items also carry a per-line `ddt_number` and a `discount_pct`, and its parties a `partita_IVA` (VAT id) alongside a top-level `ship_to_address`. The awkward field is `discount_pct`: in the ground truth it appears in three inconsistent shapes—a single object, a list of objects, or null—so a validator canonicalizes all three to one string, ensuring prediction and reference compare like for like. A second caveat is intrinsic to the corpus: `payment.method` and `expiry_date` are inferred or computed by the labeler rather than printed on the page, so on roughly half the affected invoices they are unrecoverable from the document alone and cap the achievable invoice header score uniformly across pipelines.

3.3. Two fields that set the ceiling

Two DDT line-item fields deserve emphasis because they dominate the residual error (§8). `order_id` is the *supplier's* internal order reference for a line, while `buyer_order_ref` is the *customer's* own reference; both are frequently printed once per order group—in a sub-header row above several items—rather than on every line. Correctly attributing them to each item is the single hardest part of the task.

4. EVALUATION METHODOLOGY

4.1. Field- and document-level metrics

We score predictions against ground truth with a single schema-driven evaluator that traverses the Pydantic model and never hard-codes field names. Text fields match when the normalized strings are equal or their RapidFuzz [2] `token_set_ratio` reaches 85; numeric fields match within an absolute tolerance of 0.01; null reference values are skipped, so a field absent from the ground truth neither helps nor hurts. From these per-field outcomes we compute, *per document*, the fraction of its scored cells that are correct, and aggregate as follows:

- **Field accuracy** — the mean of those per-document fractions: a macro-average that weights every document equally, so a few large multi-page DDTs cannot dominate.
- **Header accuracy** and **item accuracy** — the same macro-average restricted to header and line-item scopes.
- **Item-count match** — predicted and reference item lists have equal length.
- **Document-correct** — every scored field correct *and* the item count matches, the strictest possible bar.

The brief also asks for an always-available baseline signal that does not need labels; the harness computes the same fuzzy text-presence check against the document's own OCR text, used during development to flag fields absent from the page.

Uncertainty. Because field accuracy and document-correct are means over documents, we attach a 95% confidence interval to each by a percentile bootstrap that resamples the documents with replacement (20,000 draws, fixed seed); cells within a document are correlated—one mis-segmented layout misses many at once—so we resample whole documents, not cells. The intervals (Table 3) are what license, or withhold, claims that one pipeline beats another. We also re-score at `token_set_ratio` thresholds of 80 and 90 to confirm the rankings are not an artifact of the 85 cutoff (§8).

4.2. A space-insensitive rescue

One front-end (Docling) emits text with inter-word spaces stripped, e.g. `BUSSOLAPORTAMASCHIO` for *BUSSOLA PORTAMASCHIO*. Token-based fuzzy matching reads this as a single foreign token and fails, even though the content is correct. We therefore add a space-insensitive rescue: a field also counts as correct if the two strings match once all whitespace is removed. The rescue can only convert a near-miss into a match, never the reverse, so it does not inflate any method that already spaces its output correctly; it removes a pure tokenization artifact from the comparison. On Docling it lifts DDT field accuracy from 68.3% to 77.0% (the figure we report); on every other pipeline the rescue has no measurable effect.

4.3. Cost model

Cost is computed at score time from per-document token and page counts logged during prediction, using published rates verified in May 2026 (Appendix A). LlamaCloud bills credits at $1.25 per thousand; LlamaParse parsing costs 3 credits per page at the cost-effective tier and 10 at the agentic tier, while LlamaExtract adds an extraction tier of 5 (cost-effective) or 15 (agentic) credits *on top of* the parse tier. Gemini 3 Flash is billed at $0.50 and $3.00 per million input and output tokens; the Document AI Form Parser at $0.03 per page. Because the figure is recomputed from cached counts, a rate change updates every cost without rerunning a single document.

4.4. Latency and its caveat

Latency is wall-clock time per document for the pipeline as run. For the methods whose OCR we executed offline on a GPU (Surya, Docling, Marker; §6), the reported latency covers only the local extraction call and therefore *understates* their true end-to-end time; we flag this wherever those numbers appear. Accuracy and cost are unaffected by where the OCR ran.

5. PIPELINES BENCHMARKED

We evaluate fifteen pipelines. Twelve share a common back-end—the same Gemini model, the same extraction prompt, and the same JSON schema—and differ only in the *front-end* that turns the PDF into the model's input. The remaining three are hosted schema-guided engines that bypass the shared back-end and produce the validated JSON end to end: the cost-effective and agentic tiers of LlamaExtract, and Google's Document AI Custom Extractor configured with our schema and backed by the foundation model. We treat these as monolithic baselines. This design (Figure 1) makes the front-end the single independent variable: any accuracy difference among the twelve is attributable to the document representation, not to a different language model or prompt.

Naming. Identifiers below and in the result tables follow a short convention: the prefix names the front-end (*gemini*, *llama*, *mistral*, *surya*, *docling*, *marker*, *detr*, *google_ocr*, *docai*); a *_gem* suffix flags that the same Gemini back-end consumes the front-end's output; the LlamaParse variants use *_v* for text-only Markdown and *_vi* for Markdown-plus-image; *_agentic* marks the more expensive tier of a hosted engine.

Native vision–language (1). *Gemini* uploads the PDF and decodes JSON directly under the fixed schema, in a single pass with no self-correction and no fallback model—deliberately, so it is comparable to the front-end pipelines rather than advantaged by extra calls.

LlamaParse + Gemini (3). LlamaParse converts the PDF to Markdown, which is then handed to the same Gemini call. We test the cost-effective tier with Markdown only (*llama_gem_v*) and with the page image also attached (*llama_gem_vi*), and the agentic parse tier (*llama_agentic*).

Mistral OCR + Gemini (1). *mistral_gem* sends the PDF to Mistral's hosted document OCR (`mistral-ocr-latest` [22]), whose per-page Markdown feeds the same back-end—a commercial alternative to LlamaParse in the same Markdown-front-end role.

Local OCR + Gemini (2). Surya OCR produces reading-order text (*surya_gem*). A variant additionally runs Surya's Table Transformer-style table recognition and renders the item table as a Markdown grid (*surya_table_gem*), testing whether explicit table structure helps.

Layout converters + Gemini (2). Docling (*docling_gem*) and Marker with its agentic `use_llm` mode (*marker_gem*) emit layout-aware Markdown that feeds the same back-end.

Table detection + Gemini (1). *detr_gem* runs the Table Transformer detector, crops the detected table region, and passes the crop plus the full page to Gemini; it applies to DDTs only.

Google Document AI + Gemini (2). Two pipelines use Google's hosted document processors as front-ends to the shared back-end. *google_ocr_gem* runs the Document AI OCR processor and feeds its plain reading-order text to Gemini—the cloud-OCR analogue of *surya_gem*. *docai_form* runs the pre-trained Form Parser and feeds its key–value pairs plus OCR text. We also trialled the pre-trained Document AI *Invoice* processor and discuss its removal in §C.

Hosted schema-guided extractors (3). Three pipelines bypass the shared back-end entirely and emit the validated JSON from their own engines. *llamaextract*, in its cost-effective and agentic tiers, calls LlamaExtract with the same target JSON schema and the same instruction text as a system prompt. *docai_custom* configures a Google Document AI Custom Extractor processor with a `DocumentSchema` derived directly from our Pydantic schemas (one root entity per document type with nested `line_item` children) and runs it against the foundation-model version (`pretrained-foundation-model-v1.4-2025-02-05`, GA); we map the returned typed entities into the same Pydantic objects, with no Gemini call. These three are the only methods evaluated as monolithic baselines.

6. EXPERIMENTAL SETUP

6.1. A two-stage harness

Predictions are expensive—paid API calls, GPU OCR, multi-second model latency—while scoring is cheap arithmetic over

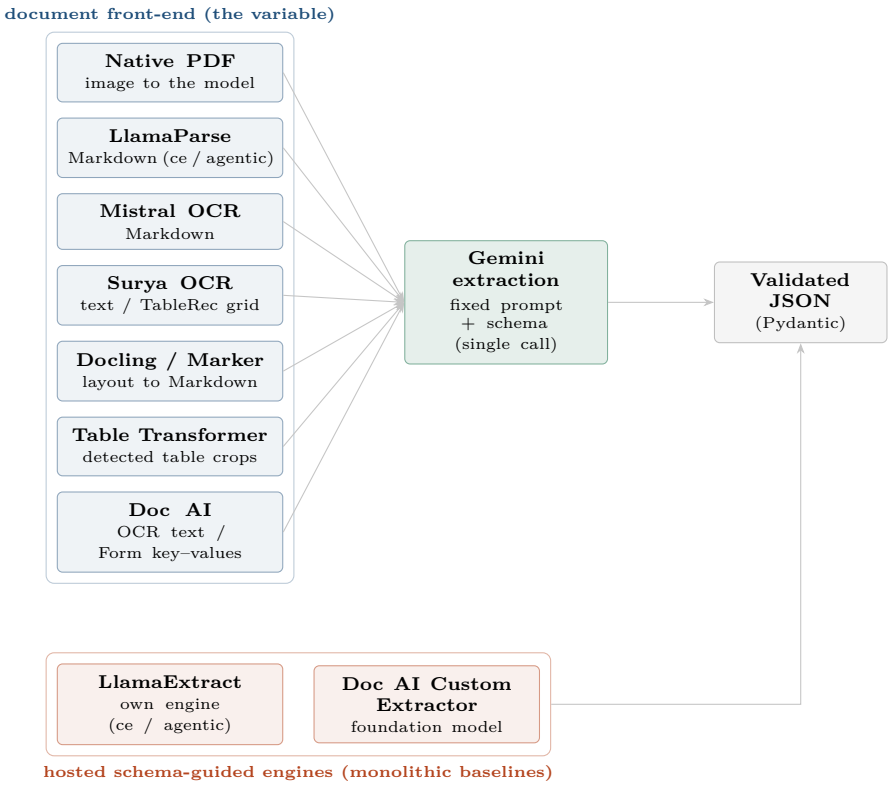


Figure 1. The benchmark holds the extraction back-end fixed and varies only the front-end. Twelve pipelines feed a single Gemini call with an identical prompt and target schema—Mistral OCR and Google's Document AI (OCR and Form Parser) among them. Three hosted schema-guided engines bypass the shared back-end and produce the validated JSON end to end: LlamaExtract (cost-effective and agentic tiers) and Google's Document AI Custom Extractor (foundation-model backed, configured with our schema).

cached outputs. The harness keeps the two apart. Stage one runs a method over the corpus and writes one prediction file per document under a method-specific cache; stage two reads those caches, runs the evaluator, and assembles the results matrix. Two properties follow directly. Re-running one method touches only its own cache, so a fix to a single pipeline never disturbs the others. And a new metric is recomputed from the cached predictions with no re-prediction at all—the cost column in this report was added this way after every pipeline had already run. Determinism is enforced with a fixed temperature and seed, so cached predictions are stable.

6.2. Two-phase offload for heavy OCR

Surya, Docling, and Marker saturate every CPU core and drove a laptop to its thermal limit; they belong on a GPU. Their language step, however, must reach the Gemini API, and the available GPU host—a shared cluster node with two NVIDIA A40s—proved unreliable for large outbound model requests. We therefore split these pipelines in two phases: the OCR or layout conversion runs on the GPU and writes its Markdown to a cache, and the identical Gemini extraction step runs locally over that cache. On the A40, Surya OCR dropped from roughly two minutes per document on CPU to about five seconds. The split is transparent to scoring—the cached text is the same regardless of where it was produced—and is the source of the latency caveat in §4. Surya's recognition predictor is not thread-safe, so its phase ran single-threaded.

6.3. Reproducibility

Every method is a registered function behind one command-line entry point; predictions, metadata, per-document scoring detail, and the result matrices are all written to disk, and the report's tables and figures are regenerated from those artifacts by a single script. The exact commands are listed in Appendix B.

7. RESULTS

Tables 1 and 2 report every pipeline on the DDT and invoice corpora; Figure 2 places them in the cost–accuracy plane. We read the outcome along three axes.

How to read the numbers. Field-accuracy figures should be read against the bootstrap confidence intervals in Table 3 (§8.5). The intervals draw one robust line and one fragile one. The leaders' field-accuracy intervals overlap heavily—"llama_gem_vi" $[90.1, 95.7]$ versus *surya_gem* $[89.5, 95.2]$, etc.—so we treat them as one tied group rather than ranking within. The structure-heavy and agentic methods sit *below* that group with non-overlapping intervals, so claims of the form "simpler representations win" are real, not sampling noise. Document-correct gaps go the same way as the field-accuracy ones but with wider intervals at n=50; on the 20-document invoice set every interval is too wide to separate methods at all. The headline numbers in this section should therefore be read as point estimates of a group rather than as a ranking.

Accuracy. On DDTs, four pipelines sit at the top within a single point of field accuracy: *llama_gem_vi* (93.1%), *llama_gem_v* (93.0%), *llama_agentic* (92.9%), and *surya_gem* (92.6%), with native *gemini* a little behind at 90.4%. As the confidence intervals in §8 (Table 3) make explicit, these top values are statistically indistinguishable—the gap among the leaders is well inside sampling noise on fifty documents—so we treat them as one leading group rather than ranking within it. The same methods lead on invoices (92.6–93.6%), where the spread narrows further because invoice tables are denser and better delimited.

Table 1. DDT corpus (50 documents). Field/header/item are field-accuracy within scope; Doc is the strict document-correct rate; Items is item-count match; Lat. is per-document latency (rows marked £ are extraction-only, excluding the offline GPU OCR step, §4); Cost is USD per document at published rates. Per-method 95% confidence intervals for Field and Doc are in Table 3.

Pipeline	Field acc.%	Doc ok%	Items match%	Header acc.%	Item acc.%	Lat. s	Cost $
Gemini (native PDF)	90.4	34.0	98.0	98.9	88.1	20.9	0.0044
LlamaParse$_{ce}$ + Gemini (md)	93.0	44.0	94.0	98.8	91.3	40.8	0.0091
LlamaParse$_{ce}$ + Gemini (md+img)	93.1	52.0	94.0	100.0	91.2	25.4	0.0093
LlamaParse$_{ag}$ + Gemini (md+img)	92.9	58.0	94.0	99.4	91.2	46.5	0.0190
Mistral OCR + Gemini	87.8	34.0	96.0	98.8	84.9	19.4	0.0060
Surya OCR + Gemini	92.6	44.0	94.0	97.5	91.5	14.7£	0.0047
Surya + TableRec + Gemini	87.6	24.0	96.0	96.1	85.3	21.2£	0.0049
Docling + Gemini	77.0	10.0	84.0	90.9	73.1	21.0£	0.0049
Marker (use_llm) + Gemini	81.5	14.0	80.0	88.7	79.8	20.0£	0.0050
Table Transformer + Gemini	82.7	12.0	88.0	93.6	80.2	18.9	0.0046
Google OCR + Gemini	90.4	34.0	96.0	96.5	88.8	16.2	0.0064
Doc AI Form + Gemini	87.2	30.0	94.0	98.1	84.1	19.8	0.0379
LlamaExtract$_{ce}$	82.7	16.0	80.0	94.7	79.9	158.8	0.0110
LlamaExtract$_{ag}$	66.3	12.0	52.0	96.7	59.1	73.0	0.0344
Doc AI Custom Extractor (e2e)	87.6	16.0	94.0	90.2	86.8	11.4	0.0330

Table 2. Invoice corpus (20 documents, two vendors). Columns as in Table 1; the Table Transformer pipeline is DDT-only and omitted.

Pipeline	Field acc.%	Doc ok%	Items match%	Header acc.%	Item acc.%	Lat. s	Cost $
Gemini (native PDF)	92.8	20.0	85.0	90.0	94.1	16.9	0.0033
LlamaParse$_{ce}$ + Gemini (md)	92.6	30.0	100.0	88.2	95.0	16.9	0.0102
LlamaParse$_{ce}$ + Gemini (md+img)	93.6	30.0	100.0	90.5	94.9	20.7	0.0104
LlamaParse$_{ag}$ + Gemini (md+img)	93.3	25.0	95.0	89.5	95.2	19.9	0.0239
Mistral OCR + Gemini	90.4	20.0	85.0	86.8	92.6	16.9	0.0058
Surya OCR + Gemini	92.7	25.0	85.0	90.5	93.3	13.3£	0.0039
Surya + TableRec + Gemini	88.1	5.0	100.0	85.5	90.5	20.0£	0.0043
Docling + Gemini	92.0	10.0	100.0	84.5	95.3	13.9£	0.0044
Marker (use_llm) + Gemini	88.2	10.0	90.0	83.2	90.0	16.0£	0.0043
Google OCR + Gemini	91.8	25.0	90.0	86.8	93.9	15.2	0.0062
Doc AI Form + Gemini	87.4	5.0	80.0	80.0	91.8	16.7	0.0507
LlamaExtract$_{ce}$	92.0	30.0	95.0	86.4	95.1	63.4	0.0155
LlamaExtract$_{ag}$	81.8	30.0	75.0	87.7	80.9	92.1	0.0484
Doc AI Custom Extractor (e2e)	84.5	0.0	100.0	81.4	86.6	7.9	0.0465

The two commercial OCR front-ends land mid-table and do not join the leaders: *google_ocr_gem* reaches 90.4% on DDTs and 91.8% on invoices—level with native *gemini*—while *mistral_gem* is lower at 87.8% and 90.4%. The strict *document-correct* rate is low everywhere (highest at 58% on DDTs for *llama_agentic*) because it demands every field and the item count to be exactly right; §8 shows the misses concentrate in two fields.

Cost. Native *gemini* is the cheapest capable option at $0.0044 per DDT, essentially the token cost of one multimodal call; *surya_gem* matches it at $0.0047 because its OCR is local and free. The LlamaParse pipelines add a parse fee to reach $0.009; the two commercial OCR front-ends sit just above them at $0.006 (*mistral_gem*) and $0.0064 (*google_ocr_gem*)—a per-page OCR fee on top of the Gemini call—without buying accuracy. The agentic and hosted-API tiers are several times dearer—*llama_agentic* $0.019 (agentic parse), *llamaextract*$_{ag}$ $0.034 (hosted extraction), *docai_form* $0.038 (a per-page Form Parser fee on top of the Gemini call)—without a commensurate accuracy gain.

The frontier. Figure 2 makes the trade-off explicit: *gemini*, *surya_gem*, and the cost-effective LlamaParse pipelines define the efficient frontier, while the structure-heavy and agentic methods—and the commercial OCR front-ends, which cost more than Surya for no accuracy gain—sit strictly inside it. We analyze why next.

8. ANALYSIS

8.1. The ceiling is two reference fields

Field accuracy stalls near 93% for the best pipelines, and the shortfall is not spread evenly. Figure 3 breaks DDT accuracy down by line-item field for four pipelines chosen to span the approaches—native vision (*gemini*), the strongest markdown front-end (*llama_gem_vi*), the strongest local OCR (*surya_gem*), and a layout converter (*docling_gem*)—all feeding the identical Gemini back-end, so the bars isolate the effect of the front-end alone. (Plotting all fifteen would be illegible; the full per-method accuracy is in Tables 1–2.) Header fields and the easy item columns—description, quantity, document number—are essentially solved by the stronger front-ends (≥ 97% for native Gemini and LlamaParse). The deficit lives almost entirely in three columns: `order_id`, `buyer_order_ref`, and to a lesser extent `code`. These are the per-line references that are often printed once per order group rather than on each row (§3); attributing them to the right item is the task's hard core.

8.2. The errors are omissions, not hallucinations

Classifying the misses is instructive. On `order_id`, native *gemini* is wrong on 102 DDT cells; of these, 84 are *drops* (the model returned null where a value exists) and *none* is a hallucination. *llama_gem_vi* both drops fewer (67) and mis-assigns fewer. The pattern explains the gap in Figure 3: reading the raw image, the model must infer that a reference in a group header applies to the rows beneath it, and it frequently declines to; LlamaParse's Markdown re-lays the table so the value sits with each row, and the same model then propagates it. The anti-hallucination instruction works as intended—the failure mode is silence, not invention—which is the safer error for a downstream reconciliation system.

8.3. Simpler representations win

The most consistent result is negative. Every front-end that imposes *more* structure underperforms clean reading-order text. Surya's Table Transformer variant scores 87.6% against plain Surya's 92.6%, because the table detector mis-segments these sparse, often borderless layouts—promoting header blocks to spurious tables and mis-aligning cells—and hands the model a confidently wrong grid. Marker's agentic Markdown (81.5%) and Docling (77.0%) reorganize the page in ways that detach references from their rows, and the Table Transformer crop pipeline (82.7%) discards header context that lives outside the detected table. The lesson is that on documents whose tables are sparse and weakly ruled, faithful linear text plus one capable model beats brittle structure recovery.

8.4. Hosted services don't lead either

The point holds from both directions—hosted OCR front-ends and hosted end-to-end engines. Mistral OCR, a paid and well-regarded engine, lands at 87.8% on DDTs and 90.4% on invoices—below plain Surya and the LlamaParse pipelines; its Markdown reflows the tables much as LlamaParse does but less faithfully on these layouts, so it buys nothing over free local OCR. Google's Document AI OCR text does better—90.4% on DDTs (matching native extraction) and 91.8% on invoices, with intervals that overlap the leading group—but it still neither surpasses local Surya, which uses the identical reading-order-text recipe at no API fee, nor earns its per-page cloud cost. Among the schema-guided engines, LlamaExtract's agentic tier is the worst pipeline overall (66.3% on DDTs, 81.8% on invoices) at three times the cost of its cost-effective tier, which itself trails the LlamaParse-plus-Gemini pipelines—the more aggressive engine moved accuracy the wrong way by over-restructuring the document. Google's Document AI Custom Extractor, configured with the same Pydantic schema (each property carrying the same field description the Gemini pipelines see through their JSON schema, so it is run with equivalent guidance) and backed by the GA foundation model, lands at 87.6% on DDTs and 84.5% on invoices for $0.033 and $0.046 per document—ahead of the LlamaExtract tiers and on a par with Doc AI Form, but still below the leading group.

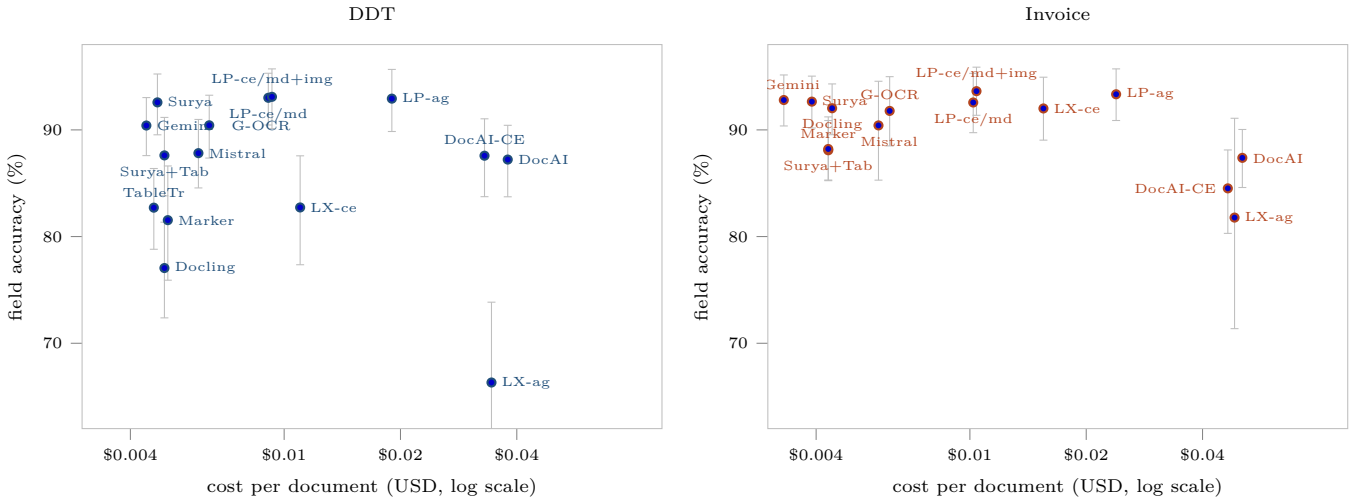


Figure 2. Cost–accuracy plane; whiskers are 95%bootstrap confidence intervals on field accuracy (resampling documents). The efficient frontier is held by native Gemini, Surya, and cost-effective LlamaParse; agentic and hosted-API tiers lie strictly inside it. The leaders' intervals overlap—they are not separable on field accuracy—while the structure-heavy methods sit clearly below them.

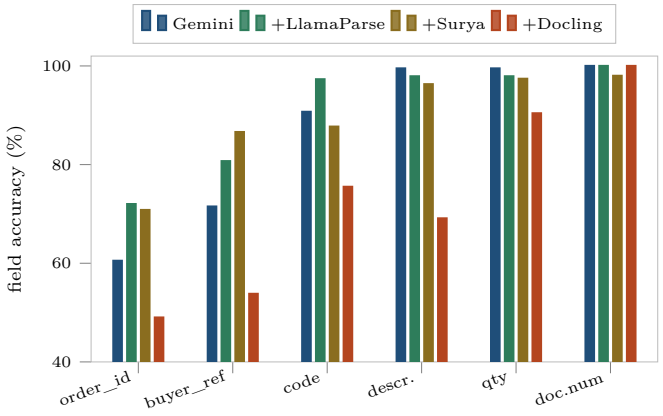


Figure 3. DDT field accuracy by line-item field. All four pipelines share the identical Gemini extraction back-end; the legend names the front-end that feeds it (*Gemini* is the native-PDF baseline, with no front-end; *+LlamaParse*, *+Surya*, *+Docling* prepend the corresponding converter). The easy columns are solved; the gap between pipelines is concentrated in the per-line order references (`order_id`, `buyer_order_ref`) and `code`.

The bottleneck is the same throughout: when the ceiling is per-line reference propagation, a stronger or schema-guided hosted engine does not move it, and paying for vendor coupling buys neither accuracy nor flexibility on this corpus.

8.5. How robust are these conclusions?

Two checks separate findings we can defend from differences that are noise. The first is the bootstrap confidence intervals of Table 3, summarised at the head of §7: the negative results are robust (structure-heavy and agentic methods sit entirely below the leading group), while the within-leader ordering and most document-correct gaps are not separable at this sample size. The recommendation therefore leans on the priority order rather than on a fragile accuracy gap (§10).

The second check is the matching threshold. Re-scoring DDTs at `token_set_ratio` thresholds of 80, 85, and 90 (Table 4) moves every field-accuracy figure by less than a point and leaves the ordering—and the leader group—unchanged, so the conclusions do not depend on the 85 cutoff.

Table 3. Field accuracy and strict document-correct rate with 95% bootstrap confidence intervals (resampling documents). The leaders' field-accuracy intervals overlap; the structure-heavy and agentic methods fall clearly below them. Invoice intervals are wide ($n = 20$).

	DDT		Invoice	
Pipeline	Field acc.%	Doc-correct%	Field acc.%	Doc-correct%
Gemini (native PDF)	90.4 [87.6, 93.0]	34.0 [22.0, 48.0]	92.8 [90.4, 95.1]	20.0 [5.0, 40.0]
LlamaParse$_{ce}$ + Gemini (md)	93.0 [90.5, 95.3]	44.0 [30.0, 58.0]	92.6 [89.7, 95.3]	30.0 [10.0, 50.0]
LlamaParse$_{ce}$ + Gemini (md+img)	93.1 [90.1, 95.7]	52.0 [38.0, 66.0]	93.6 [91.4, 95.9]	30.0 [10.0, 50.0]
LlamaParse$_{ag}$ + Gemini (md+img)	92.9 [89.9, 95.7]	58.0 [44.0, 72.0]	93.3 [90.9, 95.7]	25.0 [10.0, 45.0]
Mistral OCR + Gemini	87.8 [84.6, 91.0]	34.0 [22.0, 48.0]	90.4 [85.3, 94.6]	20.0 [5.0, 40.0]
Surya OCR + Gemini	92.6 [89.5, 95.2]	44.0 [30.0, 58.0]	92.7 [90.2, 95.0]	25.0 [10.0, 45.0]
Surya + TableRec + Gemini	87.6 [83.8, 91.2]	24.0 [12.0, 36.0]	88.1 [85.3, 90.9]	5.0 [0.0, 15.0]
Docling + Gemini	77.0 [72.4, 81.4]	10.0 [2.0, 18.0]	92.0 [89.6, 94.3]	10.0 [0.0, 25.0]
Marker (use_llm) + Gemini	81.5 [75.9, 86.6]	14.0 [6.0, 24.0]	88.2 [85.2, 91.2]	10.0 [0.0, 25.0]
Table Transformer + Gemini	82.7 [78.8, 86.4]	12.0 [4.0, 22.0]	–	–
Google OCR + Gemini	90.4 [87.4, 93.2]	34.0 [20.0, 48.0]	91.8 [88.5, 95.0]	25.0 [10.0, 45.0]
Doc AI Form + Gemini	87.2 [83.7, 90.4]	30.0 [18.0, 42.0]	87.4 [84.6, 90.0]	5.0 [0.0, 15.0]
LlamaExtract$_{ce}$	82.7 [77.4, 87.6]	16.0 [6.0, 26.0]	92.0 [89.0, 94.9]	30.0 [10.0, 50.0]
LlamaExtract$_{ag}$	66.3 [58.8, 73.9]	12.0 [4.0, 22.0]	81.8 [71.4, 91.1]	30.0 [10.0, 50.0]
Doc AI Custom Extractor (e2e)	87.6 [83.7, 91.0]	16.0 [6.0, 26.0]	84.5 [80.3, 88.1]	0.0 [0.0, 0.0]

9. EXPLAINABILITY

A production reviewer must be able to ask, of any extracted value, "where on the page did this come from?". None of the fifteen pipelines in §7 answer that question by themselves — they all return a value, but the spatial signal that would back it is either absent at the front-end (native Gemini, page-level Mistral OCR) or available upstream and *discarded* on the way to the model (Surya, Docling, Marker, LlamaParse, Document AI). This section is about that gap: which front-ends carry an explainability signal at all, at what granularity, and how much it costs to actually surface it.

9.1. What each front-end natively offers

We surveyed every predictor in the benchmark for the spatial signal its front-end *could* produce, independent of what our predictors currently consume. The result is summarized in Table 5; the granularity column reports the smallest unit the front-end emits with coordinates, and the confidence column records whether the SDK returns a numeric confidence on that unit.

Three observations follow from the table. First, *native vision–language extraction is the one front-end with no spatial signal to surface*: Gemini on the raw PDF answers "what" but not "where", and getting "where" out of it requires asking the model to self-report coordinates — a separate, unverified channel. Second, *the OCR-shaped front-ends differ sharply in granularity even though their accuracy numbers in §7 look similar*: Surya emits one bbox per OCR line by construction, Google OCR descends

Table 4. DDT field accuracy under fuzzy-match thresholds 80/85/90. The ranking is stable; the report uses 85.

Pipeline	τ=80	τ=85	τ=90
Gemini (native PDF)	90.5	90.4	90.1
LlamaParse$_{ce}$ + Gemini (md)	93.3	93.0	92.8
LlamaParse$_{ce}$ + Gemini (md+img)	93.4	93.1	92.9
LlamaParse$_{ag}$ + Gemini (md+img)	93.2	92.9	92.7
Mistral OCR + Gemini	88.5	87.8	87.6
Surya OCR + Gemini	92.9	92.6	92.4
Surya + TableRec + Gemini	87.9	87.6	87.1
Docling + Gemini	77.8	77.0	76.1
Marker (use_llm) + Gemini	82.2	81.5	81.1
Table Transformer + Gemini	83.5	82.7	82.0
Google OCR + Gemini	90.8	90.4	90.1
Doc AI Form + Gemini	87.9	87.2	86.5
LlamaExtract$_{ce}$	83.0	82.7	82.7
LlamaExtract$_{ag}$	66.9	66.3	65.7
Doc AI Custom Extractor (e2e)	87.7	87.6	87.1

Table 5. What spatial signal each front-end can produce, and whether the benchmark predictor currently surfaces it. Granularity is the smallest unit the front-end returns with coordinates; "cell" means per-table-cell, "element" means per-paragraph or per-table-as-a-whole. The right-most column is *exposed today*: does our predictor put this signal into the prediction.

Pipeline	Bbox granularity	Confidence	Exposed
Gemini (native)	none	no	–
LlamaParse$_{ce/ag}$ + Gemini	element + sub-span; table=whole	per element	no
Mistral OCR + Gemini	page (as called)	no	no
Surya OCR + Gemini	per OCR line	per line	no
Surya + TableRec + Gemini	line + cell (tables)	per line	no
Docling + Gemini	per element + table cells	no	no
Marker + Gemini	per block (JSON renderer)	no	no
Table Transformer + Gemini	whole table	per detection	no
Google OCR + Gemini	token / line / paragraph / block	per element	no
Doc AI Form + Gemini	form-field + token	per element	no
LlamaExtract$_{ce/ag}$	hosted citations (GUI)	hosted (GUI)	no*
Doc AI Custom Extractor	**per schema entity**	**per entity**	no
cited_audit (this work)	inherits Surya/LlamaParse	–	**yes**

*LlamaExtract requests citations and confidence and shows them in its hosted reviewer UI, but the SDK's `extract_metadata` returns `null` on the runs we made.

all the way to tokens, while LlamaParse's cost-effective tier collapses every cell of a table into a single whole-table rectangle — a property of the parser, not of how we call it. Third, *Doc AI Custom Extractor is the only front-end whose output is already aligned 1:1 with the target schema*: each entity it returns carries its own bbox and confidence, with no LLM-side citation pass needed. The flip side is that we paid a setup tax to train the extractor and the end-to-end accuracy on this corpus came in below the leading group (87.6% on DDTs, Table 1).

The right-most column makes the operational gap explicit: only the cited-audit pipeline we describe below actually puts a spatial signal into the prediction. Every other predictor in the benchmark consumes Markdown and returns text-only JSON — the bboxes its front-end produced are dropped before extraction.

9.2. Surfacing the signal: a two-pass citation layer over LlamaParse

To close the gap on our recommended front-end we add a thin *citation layer* on top of the production `llama_gem_vi` call. The mechanism is two Gemini calls per document. Pass 1 is bit-for-bit identical to the production extractor — same prompt, same schema, same multimodal contents over LlamaParse Markdown plus the PDF — so the values that go to the user are exactly the ones the recommended pipeline would have produced. Pass 2 receives the already-extracted values together with a *numbered index of every layout span on the page* and is asked, for each schema leaf, to return the list of span IDs whose text contributed to that leaf. Bbox attachment is then a deterministic lookup into the layout index; nothing in pass 2 can change a value, and we do not post-process the bboxes LlamaParse hands us.

Why two passes rather than one. Our first implementation folded the citation channel into the extraction call itself — a wrapped schema with a `source_ids` field on every leaf, populated in the same step that produced the value. That design measured a *2.4 pp field-accuracy regression and a *18 pp document-correct regression on the DDT corpus relative to baseline `llama_gem_vi`, with the loss concentrated in `items[*].buyer_order_ref`: the model put the right text in the *wrong* schema slot to keep its citation tidy. Splitting the call removes the pressure — pass 1 sees no spans, pass 2 cannot rewrite values — and closes the gap. On the same fifty DDTs the two-pass design lands at 93.1% field accuracy and 48% document-correct, statistically level with the baseline (92.0% / 48%); on the twenty invoices, 93.8% and 35% versus 93.1% and 35%. The explainability tax is gone.

The cost we do pay. Two Gemini calls instead of one. End-to-end latency rises from 25.4 s to 60.9 s per DDT and from 20.3 s to 57.6 s per invoice, roughly 2.5 , and the per-document API cost rises by approximately one Gemini call — from $0.009 to about $0.014 when the second call sees a prompt of comparable size to the first. Because pass 2's prompt is dominated by a numbered listing of layout spans (one short line per span; the actual text is bounded), the second call is not free but is well below the cost of an *agentic* parse, and it buys a signal the agentic tiers do not provide.

The ceiling we cannot move. The bboxes the audit UI shows are the bboxes LlamaParse hands us, by deliberate design. For header and single-line fields that gives tight rectangles around the cited span. For table cells — item code, description, quantity, unit price — the cited span is correctly identified but the attached bbox is the *entire table's outer rectangle*, because LlamaParse cost-effective does not expose per-cell coordinates. This is a property of the parser, not a defect of the citation layer. The two paths that would lift it are Surya + TableRec (per-cell polygons, local) and Doc AI Custom Extractor (per-entity bboxes by construction); the cited pipeline already accepts a Surya tier as a drop-in, and Doc AI Custom would require switching extractor entirely.

9.3. Where this leaves the deployment choice

The earlier recommendation in §10 — a LlamaParse-to-Markdown front-end feeding the production model — already used auditability of the intermediate Markdown as a tie-breaker among the statistically tied leaders. The two-pass citation layer turns that text-level auditability into per-field, spatial auditability at one extra Gemini call. It is the right setting for an audit/review surface (the FastAPI UI in `app/`) and for any human-in-the-loop validation flow that needs to point a reviewer at the source span. For batch back-of-house extraction where no reviewer will look at a single document, the one-call `llama_gem_vi` pipeline remains the right choice; the citation layer is additive, not a replacement.

10. DISCUSSION AND RECOMMENDATION

10.1. Reading the results through the priority order

The client ranks goals as generalization, then accuracy, robustness, cost, and latency. On *generalization*, every pipeline here is template-free by construction and was scored across more

than twenty DDT layouts and two invoice vendors with no per-supplier tuning. On *accuracy*, the central finding of §8.5 is that the leading pipelines—native Gemini, Surya OCR, and the LlamaParse front-ends—are statistically indistinguishable: their field-accuracy confidence intervals overlap. The strict document-correct rate leans consistently toward the layout front-ends—a LlamaParse-to-Markdown front-end gets 52% of DDTs and 30% of invoices completely right, Surya OCR 44% and 25%, native extraction 34% and 20%, with the edge concentrated in exactly the per-line order references the downstream linking stage will depend on—but on seventy documents that gap is within sampling noise, so it is a suggestive tendency rather than a decisive separation. Because accuracy does not cleanly rank the leaders, the lower-priority axes become the tie-breakers. *Robustness* favours fewer moving parts: a single model call has no parser to mis-segment a layout and no external service tier to silently change behaviour, with local Surya OCR sitting between native extraction and the hosted LlamaParse service. On *cost* and *latency* the differences are immaterial—every capable pipeline is at or below a cent per document. What the intervals *do* settle is the bottom of the table: the structure-heavy and agentic methods fall clearly below the leaders, so the priority order cleanly excludes them.

10.2. Recommendation

We recommend **a cost-effective LlamaParse-to-Markdown front-end feeding the same model as the production pipeline**. The case for it is not a significant accuracy win—it sits in a statistical tie with native Gemini and Surya at the top of the table (93.1% on DDTs, 93.6% on invoices), and its higher document-correct rate, though consistent, is within the confidence intervals (§8.5)—but the tie-breaker the priority order then reaches for: its Markdown is *auditable source text*, a layout artifact to show a reviewer and to anchor the eventual document-linking stage, and the one feature among the tied leaders that directly serves the project's second goal. The roughly one-cent-per-document parse fee is immaterial. Where external dependencies or cost must be minimized, **local Surya OCR with the same model** is an equally accurate, in-house alternative that costs about as much as native extraction. And **native Gemini on the raw PDF** is the lean fallback—the simplest to operate and the cheapest, and statistically level with the leaders on field accuracy—for settings where the auditable trail is not needed; on the present evidence it is a defensible production baseline in its own right. The agentic tiers, table-detection front-ends, and hosted invoice processors are not justified on this corpus.

10.3. Limitations

The corpus is modest (seventy documents) and drawn from a single procurement relationship, so absolute numbers should be read as indicative rather than as population estimates. The confidence intervals of §8.5 make the consequence explicit: the broad ordering is sound—the leading group is cleanly separated from the structure-heavy methods—but differences *within* the leaders, and most document-correct gaps, are not resolvable at this sample size, and the twenty-document invoice set is especially underpowered. We did not stress-test genuinely degraded scans, the worst case named in the brief. By design we also fix the extraction model to a single vision–language model (Gemini) so that the front-end is the only variable; the finding that native extraction leads therefore speaks to that model, and a multi-model comparison—other proprietary VLMs and open-weight Qwen2.5-VL [24] or InternVL [9]—would test how model-specific it is. Finally, the cost figures use list prices and exclude volume discounts and the unmetered language calls that Marker makes internally during conversion; they also book local OCR (Surya, Docling, Marker, the detector) at zero API fee, which is the right number for comparing *marginal* cost but understates the in-house option—the GPU and operational cost of self-hosting is real and would need to be weighed against the per-page parse fee in any on-premises Surya deployment.

10.4. Future work

The natural next step is the project's second goal: reconstructing the order–DDT–invoice graph and flagging item-level mismatches (price, quantity, unit of measure), which builds directly on the validated JSON produced here. Two targeted improvements would lift the present ceiling: an explicit post-processing pass that propagates a group-header reference down to its member rows, and a small amount of additional ground truth on degraded scans to characterize robustness. An on-premises variant built on a self-hosted OCR model would also be worth measuring if API use becomes constrained. On the benchmark itself, the most informative additions are the hosted invoice extractors we did not run (§C)—Azure AI Document Intelligence [17] and Amazon Textract [16], whose prebuilt invoice models may behave differently from the OCR and parsing front-ends measured here—together with at least one second extraction model to gauge model sensitivity.

11. CONCLUSION

Holding the model, prompt, and schema constant across twelve of fifteen pipelines isolates the front-end as the cause of every accuracy difference, and the answer is consistent across both document types: a lightweight layout parser, local OCR, and native vision–language extraction form a leading group that reaches roughly 93% field accuracy for about a cent per document and is, on bootstrap confidence intervals, statistically indistinguishable. Methods that impose explicit table or layout structure—and the more expensive agentic and schema-guided hosted tiers—fall clearly behind, confirming that a heavier parser does not lift the ceiling on these documents. The residual error is narrow and well understood: it lives in the per-line order references and manifests as omission rather than invention.

Because the leaders are tied on accuracy, the choice turns on the remaining priorities. For the ChatDDT deployment we recommend the cost-effective LlamaParse-to-Markdown front-end—it matches the best accuracy and uniquely yields auditable text to anchor the downstream linking stage—with local Surya OCR as its in-house, lower-cost alternative and native extraction as a lean, defensible baseline. Document linking and on-premises OCR are the clear next steps.

A. COST RATES

All costs in this report are computed from logged per-document token and page counts using the published rates below, verified May 2026 [25–27]. LlamaExtract bills an extraction tier *plus* a parse tier per page (the parse tier defaults to match the extraction tier).

Table 6. Published unit rates used by the cost model.

Item	Rate
LlamaCloud credit	$1.25 / 1000 credits
LlamaParse parse (cost-eff. / agentic)	3 / 10 credits per page
LlamaExtract extract (cost-eff. / agentic)	5 / 15 credits per page
Gemini 3 Flash input / output	$0.50 / $3.00 per 1M tokens
Document AI Form Parser	$0.03 per page
Surya / Docling / Marker / DETR OCR	local (no API fee)

B. REPRODUCTION

The benchmark is driven by one command-line entry point; predictions are cached per method, so any subset can be (re)run independently and the report regenerated from the artifacts.

```
# Stage 1: predict (cached per method)
python -m benchmarks.run --all-methods --doc-type ddt
python -m benchmarks.run --all-methods --doc-type invoice

# Stage 2: score from cache (no re-prediction)
python -m benchmarks.run --score-only --all-methods \
    --doc-type ddt --metrics full

# Regenerate the report's tables and figure data
python -m benchmarks.report_tables
```

Heavy OCR front-ends are produced in a separate phase on a GPU host (`benchmarks.parse_only`) and their Markdown cached; the local scoring run then reuses that cache (§6). Determinism is fixed by the model temperature and seed in the project configuration.

C. METHODS CONSIDERED BUT NOT ADOPTED

This section catalogues methods that sit *outside* the benchmark proper—those we tried and dropped before the full run, and those we did not run at all but consider in scope. Methods that are *in* the benchmark and simply underperform—the structure-heavy parsers, the agentic tiers—belong to the results, and are discussed in §8 rather than here.

C.1. Considered and dropped

Document AI Invoice processor. Google's pre-trained Invoice processor returned typed entities directly, but its schema maps poorly to ours: item accuracy was near 3% on the smoke set because it splits and labels line items by its own conventions, and it bills per page on top of that. We dropped it from the full run and kept the generic Form Parser (whose key–value output we feed to the shared model, *docai_form*) and the schema-guided Custom Document Extractor (*docai_custom*) instead.

C.2. In-scope peers we did not benchmark

A few obvious members of our families remain unrun, and we name them so the omission is explicit. The empirical finding of §8—that the ceiling is per-line reference propagation, not parse quality—bounds what these additions would change: adding Mistral OCR to the benchmark confirmed exactly that, landing the new front-end mid-pack rather than ahead (§7). The most informative additions are therefore the hosted invoice extractors, whose prebuilt invoice models may behave differently from the OCR and parsing front-ends measured here.

Open layout parsers. *MinerU* [11] and *PaddleOCR*/PP-Structure [12] are the open-source peers of Docling and Marker; PP-Structure additionally covers table-cell layout, overlapping with the Surya TableRec variant we benchmarked. We expect them to land in the same region as the existing Markdown front-ends for the reason just noted; their value, if added, would be in robustness, licensing (PaddleOCR is Apache 2.0, against the revenue-threshold clause on Surya's open license), and self-hosted operation rather than a new accuracy frontier.

Hosted schema-guided extractors. *Azure AI Document Intelligence* [17] and *Amazon Textract* [16] are the direct cloud peers of our Document AI and LlamaExtract baselines. Their prebuilt invoice models may fit the invoice corpus better than Doc AI's generic Form Parser, and they are the most informative additions to measure before any cross-vendor commitment.

Enterprise document intelligence (ABBYY). *ABBYY FlexiCapture* [15] is the named exception to the open-source and zero-shot hosted tools we measured: an enterprise IDP that ships prebuilt schemas for CMR consignment notes (the European cousin of DDT), invoices, and customs declarations, with semantic classification of senders, receivers, per-line order references, and item rows—i.e. exactly the layer at which our pipelines hit their ceiling (§8). It is the only tool we considered that targets that semantic layer directly rather than stopping at OCR or layout, and would be the most natural external comparison if the per-line reference ceiling becomes a hard production constraint. We did not run it for two reasons: it is enterprise-licensed with no published pricing, and it requires per-layout template/scenario configuration rather than the zero-shot regime every pipeline in this study operates in—so a trial is gated behind a sales contact and a labelling effort the present, fixed-cost study could not absorb.

OCR-centric VLMs. The brief pointed to recent OCR-centric models—*DeepSeek-OCR* [18] and *dots.ocr* [20]. Both require self-hosting GPU inference, and the analysis indicates the bottleneck on this corpus is not OCR quality (Surya already reads these pages well) but the propagation of per-line references, which a stronger OCR does not address. They remain reasonable candidates if an on-premises, no-API deployment becomes a hard requirement.

General parsing benchmarks. We position the present study against general parsing benchmarks such as OmniDocBench [23], which rank parsers on broad document corpora; we instead fix the model and isolate the front-end's marginal value on one narrow, sparse-table domain.

REFERENCES

[1] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers", in *European Conference on Computer Vision (ECCV)*, arXiv:2005.12872, 2020.

[2] M. Bachmann, *RapidFuzz: Rapid fuzzy string matching for Python*, https://github.com/rapidfuzz/RapidFuzz, Accessed 2026-05, 2021.

[3] Y. Huang, T. Lv, L. Cui, Y. Lu, and F. Wei, "LayoutLMv3: Pre-training for document AI with unified text and image masking", in *Proceedings of the 30th ACM International Conference on Multimedia (MM)*, arXiv:2204.08387, 2022.

[4] G. Kim *et al.*, "OCR-free document understanding transformer", in *European Conference on Computer Vision (ECCV)*, Donut; arXiv:2111.15664, 2022.

[5] B. Smock, R. Pesala, and R. Abraham, "PubTables-1M: Towards comprehensive table extraction from unstructured documents", in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, arXiv:2110.00061, 2022.

[6] Gemini Team, Google, "Gemini: A family of highly capable multimodal models", *arXiv preprint arXiv:2312.11805*, 2023.

[7] L. Blecher, G. Cucurull, T. Scialom, and R. Stojnic, "Nougat: Neural optical understanding for academic documents", *arXiv preprint arXiv:2308.13418*, 2023.

[8] C. Auer *et al.*, "Docling technical report", *arXiv preprint arXiv:2408.09869*, 2024.

[9] Z. Chen, J. Wu, W. Wang, *et al.*, "InternVL: Scaling up vision foundation models and aligning for generic visual-linguistic tasks", in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, arXiv:2312.14238, 2024.

[10] V. Paruchuri, *Marker: Convert PDFs and documents to markdown, JSON, and HTML*, https://github.com/datalab-to/marker, Accessed 2026-05, 2024.

[11] B. Wang, C. Xu, X. Zhao, *et al.*, *MinerU: An open-source solution for precise document content extraction*, https://github.com/opendatalab/MinerU, arXiv:2409.18839; Accessed 2026-05, 2024.

[12] PaddlePaddle, *PaddleOCR: Multilingual OCR toolkits and PP-Structure document parsing*, https://github.com/PaddlePaddle/PaddleOCR, Accessed 2026-05, 2024.

[13] S. Colvin *et al.*, *Pydantic: Data validation using Python type hints*, https://github.com/pydantic/pydantic, Accessed 2026-05, 2024.

[14] V. Paruchuri, *Surya: A multilingual document OCR toolkit (detection, recognition, layout, reading order, table recognition)*, https://github.com/datalab-to/surya, Accessed 2026-05, 2024.

[15] ABBYY, *ABBYY FlexiCapture: Intelligent document processing for logistics, invoices, and transport documents*, https://www.abbyy.com/flexicapture/, Accessed 2026-05, 2025.

[16] Amazon Web Services, *Amazon textract*, https://aws.amazon.com/textract/, Accessed 2026-05, 2025.

[17] Microsoft, *Azure AI document intelligence documentation*, https://learn.microsoft.com/azure/ai-services/document-intelligence/, Accessed 2026-05, 2025.

[18] DeepSeek-AI, *DeepSeek-OCR: Contexts optical compression*, https://github.com/deepseek-ai/DeepSeek-OCR, Accessed 2026-05, 2025.

[19] Google Cloud, *Document AI documentation*, https://cloud.google.com/document-ai/docs, Accessed 2026-05, 2025.

[20] rednote-hilab, *Dots.ocr: Multilingual document layout parsing in a single vision–language model*, https://github.com/rednote-hilab/dots.ocr, Accessed 2026-05, 2025.

[21] LlamaIndex, *Llamacloud documentation: Llamaparse and llamaextract*, https://developers.llamaindex.ai/python/cloud/, Accessed 2026-05, 2025.

[22] Mistral AI, *Mistral OCR*, https://mistral.ai/news/mistral-ocr, Accessed 2026-05, 2025.

[23] L. Ouyang, Y. Qu, H. Zhou, *et al.*, "OmniDocBench: Benchmarking diverse PDF document parsing with comprehensive annotations", in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, arXiv:2412.07626, 2025.

[24] Qwen Team, Alibaba Group, "Qwen2.5-VL technical report", *arXiv preprint arXiv:2502.13923*, 2025.

[25] Google Cloud, *Document AI pricing*, https://cloud.google.com/document-ai/pricing, Accessed 2026-05, 2026.

[26] Google, *Gemini API pricing*, https://ai.google.dev/gemini-api/docs/pricing, Accessed 2026-05, 2026.

[27] LlamaIndex, *Llamacloud pricing*, https://developers.llamaindex.ai/python/cloud/general/pricing/, Accessed 2026-05, 2026.